

Верификация и масштабирование архитектуры GRA-Swarm: от структурной ревизии противоречий к автономным исследовательским циклам

Verification and Scaling of the GRA-Swarm Architecture:
from Structural Revision of Contradictions to Autonomous Research Cycles

Автор: *указать автора*
указать организацию / независимый исследователь
указать e-mail

26 сентября 2026 г.

Аннотация

RU. В данной работе представлена математическая формализация и экспериментальная *in-silico* проверка мультиагентной архитектуры GRA-SWARM, основанной на парадигме GRA (*Godel-Reductio ad Absurdum*). Архитектура рассматривает противоречия не как шум, подлежащий усреднению, а как структурные сигналы для пересмотра гипотез, аксиом, зависимостей и доказательных шагов.

Рассматривается модель глубинного масштабирования через параметр мета-ревизии k , а также функционал, связывающий число агентов N с синергией, координационными потерями и глубиной структурной критики. Для данной феноменологической модели выводится условие исчезновения внутренних кризисных экстремумов. Представлен протокол GRA Final Verification Gate, разделяющий статический аудит артефактов и окончательную проверку proof-term через ядро LEAN.

Также предложен замкнутый исследовательский цикл: генерация задач, поиск контр-примеров, формализация, верификация, репликация и обновление обучающих данных только на формально проверенных артефактах. Работа формулирует проверяемые гипотезы для дальнейшего воспроизводимого сравнения GRA-SWARM с single-agent, self-critique, LLM Debate, tree search и formal-proof baseline.

EN. This work presents a mathematical formalization and an *in-silico* evaluation framework for the GRA-SWARM multi-agent architecture based on the GRA (*Godel-Reductio ad Absurdum*) paradigm. The architecture treats contradictions not as noise to be averaged away, but as structural signals that trigger revision of hypotheses, axioms, dependencies, and proof steps.

We study deep scaling through a meta-revision parameter k , together with a functional relating the number of agents N to synergy, coordination losses, and the depth of structural criticism. For this phenomenological model, a condition for the disappearance of internal crisis extrema is derived. We introduce the GRA Final Verification Gate, which separates static artifact auditing from final proof-term validation by the LEAN kernel.

Finally, we propose a closed research cycle: problem generation, counterexample search, formalization, verification, replication, and training-data updates restricted to formally verified artifacts. The paper formulates testable hypotheses for reproducible comparisons between GRA-SWARM, single-agent systems, self-critique, LLM Debate, tree search, and formal-proof baselines.

Ключевые слова: GRA, GRA-SWARM, мультиагентные системы, формальная верификация, LEAN, доказательство от противного, поиск контрпримеров, математический ИИ, воспроизводимость.

Keywords: GRA, GRA-SWARM, multi-agent systems, formal verification, LEAN, proof by contradiction, counterexample search, mathematical AI, reproducibility.

1 Введение / Introduction

1.1 Проблема согласованной ошибки / The Shared-Error Problem

RU. Современные языковые модели и мультиагентные системы часто повышают качество за счёт масштабирования вычислений, данных, контекста или числа агентов. Однако увеличение числа агентов не устраняет общую ошибку. Если несколько агентов наследуют одинаковую неверную предпосылку, голосование может только усилить уверенность в ошибочном выводе:

$$N \uparrow \not\Rightarrow \text{Truth} \uparrow. \quad (1)$$

Поэтому консенсус агентов нельзя отождествлять с истинностью:

$$\text{Consensus} \neq \text{Truth}. \quad (2)$$

EN. Modern language models and multi-agent systems often improve quality by scaling computation, data, context, or the number of agents. However, increasing the number of agents does not eliminate a shared error. If several agents inherit the same false premise, voting may only make the erroneous conclusion more confident:

$$N \uparrow \not\Rightarrow \text{Truth} \uparrow. \quad (3)$$

Therefore, agent consensus cannot be equated with truth:

$$\text{Consensus} \neq \text{Truth}. \quad (4)$$

1.2 Идея GRA / The GRA Idea

RU. Методология GRA объединяет идею доказательства от противного (*reductio ad absurdum*) с итеративной структурной ревизией. Вместо усреднения противоречий система пытается:

- 1) локализовать источник противоречия;
- 2) найти скрытое допущение, логический разрыв или конфликт ограничений;
- 3) сгенерировать варианты структурного пересмотра;
- 4) найти контрпример либо формализовать доказательство;
- 5) повторять цикл до достижения проверяемого результата.

EN. The GRA methodology combines proof by contradiction (*reductio ad absurdum*) with iterative structural revision. Instead of averaging contradictions, the system attempts to:

- 1) localize the source of a contradiction;
- 2) identify a hidden assumption, logical gap, or constraint conflict;
- 3) generate structural revision candidates;
- 4) find a counterexample or formalize a proof;
- 5) repeat the cycle until a checkable result is obtained.

2 Математическая модель / Mathematical Model

2.1 Функционал интеллекта роя / Swarm Intelligence Functional

RU. Рассмотрим феноменологическую модель полезности или “интеллекта” роя, зависящую от числа агентов N и глубины мета-ревизии k :

$$I(N, k) = \alpha_0(1 + r_{\text{gr}})^k N^2 - \beta_0 e^{-\lambda k} N^{3/2} + \gamma_0 N. \quad (5)$$

Здесь:

- $\alpha_0(1 + r_{\text{gr}})^k N^2$ — модель потенциальной синергии;
- $\beta_0 e^{-\lambda k} N^{3/2}$ — модель координационных потерь, противоречий и энтропийной нагрузки;
- $\gamma_0 N$ — линейный базовый вклад агентов;
- $r_{\text{gr}} > 0$ — темп роста эффективности структурной ревизии;
- $\lambda > 0$ — темп уменьшения штрафа противоречий;
- $k \in \mathbb{N}_0$ — глубина мета-ревизии.

Данная формула является **модельной гипотезой**, а не универсальным законом поведения реальных ИИ-систем. Её параметры должны оцениваться экспериментально в контролируемых бенчмарках.

EN. Consider a phenomenological model of swarm utility or “intelligence”, depending on the number of agents N and the depth of meta-revision k :

$$I(N, k) = \alpha_0(1 + r_{\text{gr}})^k N^2 - \beta_0 e^{-\lambda k} N^{3/2} + \gamma_0 N. \quad (6)$$

The formula is a **model hypothesis**, rather than a universal law of real AI-system behavior. Its parameters must be estimated empirically on controlled benchmarks.

2.2 Кризисные экстремумы / Crisis Extrema

Положим:

$$x = \sqrt{N}, \quad N = x^2, \quad x > 0. \quad (7)$$

Тогда функционал (5) принимает вид:

$$I(x, k) = \alpha_k x^4 - \beta_k x^3 + \gamma_0 x^2, \quad (8)$$

где:

$$\alpha_k = \alpha_0(1 + r_{\text{gr}})^k, \quad \beta_k = \beta_0 e^{-\lambda k}. \quad (9)$$

Производная по x :

$$\frac{\partial I}{\partial x} = 4\alpha_k x^3 - 3\beta_k x^2 + 2\gamma_0 x. \quad (10)$$

Для $x > 0$ внутренние критические точки определяются уравнением:

$$4\alpha_k x^2 - 3\beta_k x + 2\gamma_0 = 0. \quad (11)$$

Дискриминант уравнения (11) равен:

$$D_k = 9\beta_k^2 - 32\alpha_k\gamma_0. \quad (12)$$

После подстановки параметров:

$$D_k = 9\beta_0^2 e^{-2\lambda k} - 32\alpha_0\gamma_0(1 + r_{\text{gr}})^k. \quad (13)$$

2.3 Теорема устранения кризисных экстремумов / Crisis-Extrema Elimination Theorem

Теорема. Пусть

$$\alpha_0 > 0, \quad \beta_0 > 0, \quad \gamma_0 > 0, \quad \lambda > 0, \quad r_{\text{gr}} > 0. \quad (14)$$

Тогда существует такое конечное k^* , что при всех $k \geq k^*$:

$$D_k < 0. \quad (15)$$

Следовательно, уравнение (11) не имеет вещественных решений, а данная модель не имеет внутренних положительных критических точек по переменной x .

Доказательство. Первый член дискриминанта (13):

$$9\beta_0^2 e^{-2\lambda k} \rightarrow 0 \quad \text{при } k \rightarrow \infty. \quad (16)$$

Второй член по модулю растёт:

$$32\alpha_0\gamma_0(1 + r_{\text{gr}})^k \rightarrow +\infty \quad \text{при } k \rightarrow \infty. \quad (17)$$

Следовательно:

$$D_k \rightarrow -\infty. \quad (18)$$

По определению предела существует конечное k^* , после которого $D_k < 0$. $\square$

Ограничение интерпретации. Теорема доказывает свойство функции (5) при заданных предположениях. Она не доказывает, что реальные мультиагентные системы автоматически теряют кризисные режимы при росте k . Для такого вывода необходима эмпирическая идентификация параметров, внешняя валидация и воспроизводимые эксперименты.

2.4 Численный пример / Numerical Example

Для параметров:

$$\alpha_0 = 1.0, \quad \beta_0 = 1.5, \quad \gamma_0 = 0.5, \quad r_{\text{gr}} = 0.2, \quad \lambda = 0.5 \quad (19)$$

дискриминант равен:

$$D_k = 20.25e^{-k} - 16(1.2)^k. \quad (20)$$

При $k = 0$:

$$D_0 = 20.25 - 16 = 4.25 > 0. \quad (21)$$

При $k = 1$:

$$D_1 = 20.25e^{-1} - 16 \cdot 1.2 \approx 7.45 - 19.20 \approx -11.75 < 0. \quad (22)$$

Следовательно, для целочисленной глубины $k \in \mathbb{N}_0$ кризисные экстремумы исчезают уже при:

$$k^* = 1. \quad (23)$$

Непрерывное решение уравнения $D_k = 0$ находится численно примерно в интервале:

$$0 < k^* < 1. \quad (24)$$

3 Функционал противоречий GRA / GRA Contradiction Functional

3.1 Локальные и межуровневые противоречия / Local and Cross-Level Contradictions

Пусть S — состояние исследовательской системы, а $\mathbf{c}(S) = \{c_i(S)\}$ — набор локальных метрик противоречий. Определим общий функционал:

$$J(S) = \sum_i w_i c_i(S)^2 + \sum_{(i,j) \in E} \lambda_{ij} \text{dist}(S_i, \mathcal{L}_{j \rightarrow i} S_j)^2. \quad (25)$$

Здесь:

- $w_i \geq 0$ — вес локального противоречия;
- E — множество связей между уровнями;
- $\mathcal{L}_{j \rightarrow i}$ — оператор переноса представления между уровнями;
- $\text{dist}(\cdot, \cdot)$ — метрика рассогласования.

В математическом исследовательском контуре уровни могут соответствовать:

$$\text{определения} \rightarrow \text{леммы} \rightarrow \text{доказательные шаги} \rightarrow \text{теорема} \rightarrow \text{формальный артефакт}. \quad (26)$$

3.2 Расширенный функционал качества / Extended Quality Functional

Для генерации и решения задач удобно использовать функционал:

$$J_{\text{total}}(P, \pi) = \lambda_1 J_{\text{contr}} + \lambda_2 J_{\text{formal}} + \lambda_3 J_{\text{novelty}} + \lambda_4 J_{\text{difficulty}} + \lambda_5 J_{\text{repro}} + \lambda_6 J_{\text{leak}}, \quad (27)$$

где P — математическая задача, а π — её кандидатное решение или доказательная траектория.

Компоненты определяются концептуально следующим образом:

$$J_{\text{contr}} = \sum_i w_i c_i(P, \pi)^2, \quad (28)$$

$$J_{\text{formal}} = \mathbb{I}\{\text{формальный proof assistant не принял артефакт}\}, \quad (29)$$

$$J_{\text{novelty}} = \text{Similarity}(P, \mathcal{K}), \quad (30)$$

$$J_{\text{difficulty}} = |D_{\text{target}} - D_{\text{observed}}|, \quad (31)$$

$$J_{\text{repro}} = \text{Var}(R_1, \dots, R_m), \quad (32)$$

$$J_{\text{leak}} = \text{Evidence}(\text{готовое решение присутствовало в данных}). \quad (33)$$

4 Экспериментальная методология / Experimental Methodology

4.1 Режимы сравнения / Evaluation Regimes

Для честного сравнения необходимо фиксировать вычислительный бюджет, набор задач, модели, промпты, версии библиотек и критерии успеха.

Таблица 1: Сравнимые режимы / Compared regimes

Режим	Агенты	Глубина	Принцип
Single-agent	1	0	Один генератор решения без независимой критики
Self-critique	1	k	Один агент с повторной самопроверкой
LLM Debate	N	0	Несколько агентов, обсуждение и агрегирование ответов
GRA-Swarm	N	k	Роли генератора, контрпримерщика, аудитора аксиом, формализатора и координатора
Formal-proof baseline	N	k	Поиск с обязательной проверкой в proof assistant

4.2 Метрики / Metrics

Основные метрики должны включать:

$$\text{Accuracy} = \frac{\text{число корректных решений}}{\text{число задач}}, \quad (34)$$

$$\text{FormalRate} = \frac{\text{число решений, принятых proof assistant}}{\text{число предложенных решений}}, \quad (35)$$

$$\text{CounterexampleRate} = \frac{\text{найденные контрпримеры}}{\text{ложные гипотезы}}, \quad (36)$$

$$\text{CostPerProof} = \frac{\text{вычислительные затраты}}{\text{число формально проверенных доказательств}}. \quad (37)$$

4.3 Пример симулируемых сценариев / Example Simulated Scenarios

Следующая таблица является шаблоном для публикации результатов. Числа должны сопровождаться исходным кодом, random seed, точным описанием симулятора и доверительными интервалами.

4.4 Статистическая проверка / Statistical Testing

Для долей успеха рекомендуется публиковать доверительные интервалы Уилсона. Для сравнения двух методов на одних и тех же задачах может использоваться парный критерий Макнемара. Однако значение p -value не заменяет публикацию исходных логов, кода, бюджетов, модели угроз и результатов независимой репликации.

Таблица 2: Пример структуры отчёта по in-silico экспериментам

Эксперимент	Конфигурация	Метрика	Назначение
Consensus Trap	80% агентов имеют ложную аксиому	Accuracy, CounterexampleRate	Проверка устойчивости к ложному большинству
Fixed-budget scaling	Фиксированный бюджет B	Accuracy, CostPerProof	Сравнение масштабирования N и k
Formalization gate	Кандидаты Lean-доказательств	FormalRate, rejection reasons	Проверка различия между аудитом и kernel validation
Hidden-assumption test	Скрытая сильная аксиома	AxiomAuditRate	Обнаружение недопустимых предпосылок
Replication test	Разные модели и random seed	Reproducibility	Проверка устойчивости результата

5 GRA Final Verification Gate

5.1 Назначение протокола / Protocol Purpose

RU. GRA Final Verification Gate разделяет две разные процедуры:

- 1) статический аудит структуры артефакта;
- 2) окончательную формальную проверку proof-term через kernel proof assistant.

Статический аудит полезен, но не заменяет проверку компилятором. Он способен обнаруживать подмену заголовка, запрещённые конструкции, отсутствующее тело доказательства и изменение файлов после фиксации хэшей. Он не способен самостоятельно доказать корректность proof-term.

EN. The GRA Final Verification Gate separates two distinct procedures:

- 1) static artifact-structure auditing;
- 2) final proof-term verification by a proof-assistant kernel.

Static auditing is useful but cannot replace compiler-level verification. It can detect a substituted theorem header, forbidden constructs, missing proof bodies, and post-freeze file modifications. It cannot independently establish the correctness of a proof term.

5.2 Проверяемые условия / Verification Conditions

- 1) Совпадение утверждения со замороженной спецификацией.
- 2) Фиксация SHA-256 хэшей спецификации, кандидата и манифеста.
- 3) Запрет конструкций `sorry`, `admit`, пользовательских `axiom`, `unsafe`.

- 4) Наличие явного тела доказательства `:= by ....`
- 5) Проверка воспроизводимости: файлы не меняются после фиксации хэшей.
- 6) Компиляция в LEAN в закреплённой среде.
- 7) Аудит аксиом, например через `#print axioms target_statement`.

5.3 Статусы / Status Codes

Таблица 3: Интерпретация статусов верификации

Статус	Интерпретация
FAILED_STATIC_AUDIT	Нарушена структура артефакта, найдено запрещённое выражение, несовпадение утверждения или нарушение целостности
PROVISIONAL_PASS_REQUIRES_LEAN_KERNEL_CHECK	Кернел-чек проверки пройдены, но proof-term не был принят или не был проверен ядром Lean
FORMALLY_VERIFIED	Артефакт компилируется в закреплённой Lean-среде, а аудит аксиом не выявил запрещённых зависимостей
REPLICATION_PENDING	Формальная проверка пройдена, но независимая репликация ещё не завершена
INDEPENDENTLY_REPLICATED	Результат воспроизведён независимой средой, командой или стеком проверки

5.4 Пример протокольного вывода / Example Protocol Output

RUN ID: <timestamp>

```
PASS | StatementMatchAgent
PASS | ForbiddenConstructAgent
PASS | ProofBodyAgent
PASS | ReproducibilityAgent
```

```
Lean available: False
Lean compiled: False
```

```
FINAL VERDICT:
PROVISIONAL_PASS_REQUIRES_LEAN_KERNEL_CHECK
```

Такой статус означает только то, что статический аудит пройден. Он не является формальным принятием доказательства.

6 Автономный исследовательский цикл / Autonomous Research Cycle

6.1 Представление математической задачи / Mathematical Problem Representation

Определим задачу как кортеж:

$$P = (\mathcal{A}, \mathcal{S}, \mathcal{C}, \mathcal{M}), \quad (38)$$

где:

- $\mathcal{A}$ — набор аксиом, определений и допущений;
- $\mathcal{S}$ — пространство допустимых объектов;
- $\mathcal{C}$ — утверждение, гипотеза или цель;
- $\mathcal{M}$ — процедура проверки результата.

Задача допускается в исследовательский пул только при выполнении:

$$\text{Valid}(P) = \text{WellFormed}(P) \wedge \text{Consistent}(P) \wedge \text{NonTrivial}(P) \wedge \text{Formalizable}(P). \quad (39)$$

6.2 Роли агентов / Agent Roles

Таблица 4: Функциональные роли в GRA-Swarm

Роль	Функция
Генератор гипотез	Создаёт гипотезы, леммы, инварианты и альтернативные постановки задач
Контрпримерщик	Ищет объект, нарушающий универсальное утверждение или скрытое условие
Символьный критик	Выявляет логические разрывы, круговые рассуждения и неявные переходы
Аудитор аксиом	Проверяет, не используется ли сильная или скрытая предпосылка
Формализатор	Переводит постановку и доказательство в Lean, Coq, Isabelle или иной proof assistant
Агент воспроизводимости	Запускает независимую проверку с другим стеком, random seed или моделью
Координатор GRA	Ведёт граф зависимостей, агрегирует метрики и выбирает ветви с минимальным проверяемым J_{total}

6.3 Цикл обучения / Learning Cycle

Автономный цикл задаётся последовательностью:

$$\text{generation} \rightarrow \text{counterexample search} \rightarrow \text{structural revision} \rightarrow \text{formalization} \rightarrow \text{kernel verification} \rightarrow \text{independent} \quad (40)$$

Набор данных должен разделяться на три части:

$$\mathcal{D} = \mathcal{D}_{\text{verified}} \cup \mathcal{D}_{\text{provisional}} \cup \mathcal{D}_{\text{refuted}}. \quad (41)$$

Обновление параметров или адаптеров разрешается только на проверенных артефактах:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\text{proof}}(\theta_t; \mathcal{D}_{\text{verified}}). \quad (42)$$

Минимальное правило допуска к обновлению:

$$\Delta\theta \neq 0 \implies \begin{cases} \text{LeanCheck} = 1, \\ \text{AxiomAudit} = 1, \\ \text{CounterexampleFree} = 1, \\ \text{IndependentReplication} = 1. \end{cases} \quad (43)$$

6.4 Предотвращение замкнутой ошибки / Preventing Self-Confirmation Loops

Без внешнего верификационного контура возможен опасный цикл:

model $\rightarrow$ creates a task $\rightarrow$ creates a false solution $\rightarrow$ trains on the false solution $\rightarrow$ amplifies the error. (44)

Поэтому необходимы:

- независимый поиск контрпримеров;
- proof assistant с проверяемым kernel;
- раздельное хранение проверенных и непроверенных артефактов;
- скрытые тестовые задачи;
- независимая репликация;
- фиксированные вычислительные бюджеты;
- публикация логов, моделей и конфигураций.

7 Практические применения и ограничения / Applications and Limitations

7.1 Практические применения / Practical Applications

- 1) **Формальное математическое исследование.** Система может помогать генерировать леммы, искать контрпримеры, формализовать известные результаты и выявлять доказательные пробелы.
- 2) **Верифицированная разработка программного обеспечения.** GRA-подход применим к поиску конфликтов между спецификацией, кодом, тестами, инвариантами и требованиями безопасности.
- 3) **Научные модели и симуляции.** Можно использовать противоречия между предсказаниями и данными как сигналы для пересмотра модели, а не только для подбора параметров.
- 4) **Робототехника и автономные системы.** Архитектура может использоваться для проверки противоречий между сенсорными данными, картой среды, целями, ограничениями безопасности и планом действий.

7.2 Ограничения / Limitations

- 1) Внутреннее снижение функционала J не является доказательством объективной истины.
- 2) Исчезновение экстремумов в модели (5) не доказывает исчезновение кризисов в реальных системах.
- 3) Статический аудит не равен формальной проверке proof-term.
- 4) Формальная проверка доказательства не гарантирует, что формальная теорема соответствует содержательной научной или математической проблеме.
- 5) Генерация новых задач не устанавливает их значимость или уровень сложности без независимой экспертной оценки.
- 6) Автоматическое обучение на собственных выводах требует строгого контроля происхождения данных, проверки утечек и независимой репликации.

7.3 Корректная научная позиция / Appropriate Scientific Claim

Корректный вывод работы формулируется следующим образом:

GRA-SWARM является исследовательской архитектурой для генерации, атаки, структурного пересмотра и формальной проверки гипотез. Она не гарантирует автоматического решения открытых задач математики и не заменяет независимую экспертизу. Её ценность должна оцениваться по воспроизводимым бенчмаркам, формально проверенным артефактам, затратам вычислений, способности находить контрпримеры и результатам независимой репликации.

8 Заключение / Conclusion

RU. В работе описана архитектура GRA-SWARM, объединяющая многоагентный поиск, структурную критику, поиск контрпримеров, аудит аксиом, формализацию и независимую верификацию. Параметр глубины мета-ревизии k позволяет формализовать гипотезу о том, что дополнительные уровни критики могут быть полезнее простого увеличения числа одинаковых агентов.

Для феноменологической модели выведено условие исчезновения внутренних кризисных экстремумов. Также предложен GRA Final Verification Gate, который честно различает статический аудит и окончательную проверку ядром proof assistant. Главный практический критерий будущего успеха архитектуры — не декларация об “абсолютной истине”, а стабильное получение новых воспроизводимых и формально проверенных результатов при фиксированном бюджете ресурсов.

EN. This work described the GRA-SWARM architecture, combining multi-agent search, structural criticism, counterexample search, axiom auditing, formalization, and independent verification. The meta-revision-depth parameter k formalizes the hypothesis that additional levels of criticism may be more useful than merely increasing the number of similar agents.

For the phenomenological model, we derived a condition under which internal crisis extrema disappear. We also proposed the GRA Final Verification Gate, which honestly distinguishes static auditing from final proof-assistant-kernel validation. The key practical criterion for future success is not a declaration of “absolute truth,” but the reliable production of new reproducible and formally verified results under fixed computational budgets.

Благодарности / Acknowledgements

Авторы благодарят участников открытого обсуждения формальной верификации, много-агентных систем и математического ИИ.

Список литературы

- [1] The Lean Community. *The Lean Theorem Prover and Mathlib Documentation*. <https://leanprover-community.github.io/>
- [2] The Coq Development Team. *The Coq Proof Assistant*. <https://coq.inria.fr/>
- [3] The Isabelle Development Team. *The Isabelle Proof Assistant*. <https://isabelle.in.tum.de/>
- [4] GRA-Core-new. *Unified Hierarchical Stability Library*. Проектная документация и исходные материалы.
- [5] Kevin Buzzard. *The Formalisation of Mathematics*. Материалы по формализации математики и proof assistants.
- [6] Обзорные работы по multi-agent debate, self-critique, tree search и формальной верификации рассуждений. Конкретные источники следует дополнить в соответствии с используемыми моделями, бенчмарками и экспериментальными протоколами.